

Torch

1. torch.index_select()

1. Example, usage in NMS, a voting algorithm to remove bounding boxes with IoU greater than a threshold t with a bounding box with the biggest confidence score.

```
def non_maximum_suppression(bboxes, scores, threshold=0.5):
    """
    Input:
    1. bboxes: (n, 4): 4: [x1, y1, x2, y2] left top and right bottom.
    2. scores: (n, ): confidence score
    3. threshold: int: delete bounding box with IoU greater than threshold.
    Return:
    A long int tensor whose size is (n, )
    """
    x1 = bboxes[:, 0]
    x2 = bboxes[:, 1]
    x3 = bboxes[:, 2]
    x4 = bboxes[:, 3]
    order = scores.argsort()
    areas = (x2 - x1) * (y2 - y1)
    keep = []
    while len(order)>0:
        idx = order[-1]
        keep.append(idx)
        # Sanity check
        if len(order) == 0:
            break
        xx1 = torch.index_select(x1, dim=0, index=order)
        yy1 = torch.index_select(y1, dim=0, index=order)
        xx2 = torch.index_select(x2, dim=0, index=order)
        yy2 = torch.index_select(y2, dim=0, index=order)
        max_x1 = torch.max(xx1, x1[idx])
        max_y1 = torch.max(yy1, y1[idx])
        min_x2 = torch.min(xx2, x2[idx])
        min_y2 = torch.min(yy2, y2[idx])
        w = (min_x2 - max_x1).clamp(0)
        h = (min_y2 - max_y1).clamp(0)
        inter = w * h
        rem_areas = torch.index_select(areas, dim=0, index=order)
        union = rem_areas + areas[idx] - inter
        iou = inter / union
        maxk = iou < threshold
        order = order[mask]

    return torch.tensor(keep, dtype=torch.long)
```

2. torch.gather()

3. torch.tensor.detach().cpu().numpy()

4. device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

5. gradient clipping simple solution to gradient explosion, works well practically :

- torch.nn.utils.clip_grad_norm_(parameters, max_norm, norm_type=2)
- Example: torch.nn.utils.clip_grad_norm_(retinanet.parameters(), 0.1)

6. torch.bmm

1. torch.bmm is a special case of torch.matmul where both the tensors are 3-dimensional and contains equal number of matrices.
2. Example: torch.bmm used in neural style transfer.
3. Code:

```
def gram_matrix(features, normalize=True):
    """
    Compute the Gram matrix from features.

    Inputs:
    - features: PyTorch Variable of shape (N, C, H, W) giving features for
      a batch of N images.
    - normalize: optional, whether to normalize the Gram matrix
      If True, divide the Gram matrix by the number of neurons (H * W * C)

    Returns:
    - gram: PyTorch Variable of shape (N, C, C) giving the
      (optionally normalized) Gram matrices for the N input images.
    """
    #####
    # TODO: Implement content loss function
    # Please pay attention to use torch tensor math function to finish it.
    # Otherwise, you may run into the issues later that dynamic graph is broken
    # and gradient can not be derived.
    #
    # HINT: you may find torch.bmm() function is handy when it comes to process
```

```

# matrix product in a batch. Please check the document about how to use it. #
#####
N, C, H, W = features.shape
# (N, C, H, W) to (N, C, H*W)
features = features.view(N, C, -1)
# Create a copy of size (N, H*W, C)
features_copy = features.transpose(2, 1)
# (N, C, H*W) @ (N, H*W, C) = (N, C, C)
gram_matrix = torch.bmm(features, features_copy)
if normalize:
    gram_matrix = gram_matrix / (H*W*C)
return gram_matrix

#####
#                                     END OF YOUR CODE                                     #
#####

```

7. torch.expand

1. Example from Yolov1 loss function, compute IOU function:

```

def compute_iou(self, box1, box2):
    """ compute IOU between boxes
        - box1 (bs, 4) 4: [x1, y1, x2, y2] left top and right bottom
        - box2 (bs, 4) 4: [x1, y1, x2, y2] left top and right bottom
    """
    N = box1.size(0)
    M = box2.size(0)

    lt = torch.max(
        box1[:, :2].unsqueeze(1).expand(N, M, 2), # [N,2] -> [N,1,2] -> [N,M,2]
        box2[:, :2].unsqueeze(0).expand(N, M, 2), # [M,2] -> [1,M,2] -> [N,M,2]
    )

    rb = torch.min(
        box1[:, 2:].unsqueeze(1).expand(N, M, 2), # [N,2] -> [N,1,2] -> [N,M,2]
        box2[:, 2:].unsqueeze(0).expand(N, M, 2), # [M,2] -> [1,M,2] -> [N,M,2]
    )

    wh = rb - lt # [N,M,2]
    wh[wh < 0] = 0 # clip at 0
    inter = wh[:, :, 0] * wh[:, :, 1] # [N,M]

    area1 = (box1[:, 2] - box1[:, 0]) * (box1[:, 3] - box1[:, 1]) # [N,]
    area2 = (box2[:, 2] - box2[:, 0]) * (box2[:, 3] - box2[:, 1]) # [M,]
    area1 = area1.unsqueeze(1).expand_as(inter) # [N,] -> [N,1] -> [N,M]
    area2 = area2.unsqueeze(0).expand_as(inter) # [M,] -> [1,M] -> [N,M]

    iou = inter / (area1 + area2 - inter)
    return iou

```

2. Simpler example:

Code:

```

x = torch.rand(1, 3)
N = 5
x = x.expand(N, 3)
print(x)

```

Output:

```

tensor([[0.3884, 0.0457, 0.4364],
        [0.3884, 0.0457, 0.4364],
        [0.3884, 0.0457, 0.4364],
        [0.3884, 0.0457, 0.4364],
        [0.3884, 0.0457, 0.4364]])

```

8. torch.nn.Embedding VS torch.nn.Linear

1. torch.nn.Embedding is a lookup table. it works the same as torch.tensor but with a few twists (like possibility of using sparse embedding or default value at specified index).

Code:

```

import torch

# nn.Embedding
N, T, V, D = 2, 3, 4, 5
embedding = torch.nn.Embedding(V, D)
caption = torch.randint(0, V, (N, T))
x = embedding(caption)
print(x.shape)
print(N, T, D)
print(x)

```

Output:

```

torch.Size([2, 3, 5])
2 3 5
tensor([[[[-0.3991, -0.5553,  0.6418, -1.2668, -0.3077],
          [-0.3991, -0.5553,  0.6418, -1.2668, -0.3077],
          [-0.3991, -0.5553,  0.6418, -1.2668, -0.3077]],

         [[[-0.4294, -0.3665, -0.1000, -1.1245,  0.3728],
          [-0.3991, -0.5553,  0.6418, -1.2668, -0.3077],
          [-0.3991, -0.5553,  0.6418, -1.2668, -0.3077]]]])
grad_fn=<EmbeddingBackward0>)

```

2. `torch.nn.Linear` (without bias) is a `torch.Tensor` (weight) but it does operation on the input and the weight, which is essentially `output = input.matmul(weight, t())`:

Code:

```

# nn.Linear
N, D1, D2 = 2, 3, 4
x1 = torch.rand(N, D1)
linear = torch.nn.Linear(D1, D2)
x2 = linear(x1)
print(x2.shape)
print(N, D2)
print(x2)

```

Output:

```

torch.Size([2, 4])
2 4
tensor([[[-0.0952, -0.2378,  0.1940,  0.4159],
         [-0.4102,  0.2701,  0.7802,  0.5189]]], grad_fn=<AddmmBackward0>)

```

9. `torch.Tensor.unsqueeze`

Code:

```

a = torch.rand(5, 1)
b = torch.rand(3)
# Using * for outer product
c = a * b
print(c)

d = a @ b.unsqueeze(0)
print(d)

print((c == d).all())

```

Output:

```

tensor([[0.5483, 0.3419, 0.4980],
        [0.3854, 0.2403, 0.3500],
        [0.7277, 0.4538, 0.6610],
        [0.2675, 0.1668, 0.2430],
        [0.4835, 0.3015, 0.4391]])
tensor([[0.5483, 0.3419, 0.4980],
        [0.3854, 0.2403, 0.3500],
        [0.7277, 0.4538, 0.6610],
        [0.2675, 0.1668, 0.2430],
        [0.4835, 0.3015, 0.4391]])
tensor(True)

```

10. `torch.Tensor.unsqueeze(1)` VS `x[:, None]`

1. `[None, ...]` is numpy notation for `.unsqueeze(0)`. The two are equivalent. The version with advanced indexing might be slower because it has more checking to do to find out exactly what you want to do.

Code:

```

x = torch.rand(3, 4, 5)
print(type(x))
a = x.unsqueeze(1)
b = x[:, None]
print((a==b).all())

```

Output:

```

<class 'torch.Tensor'>
tensor(True)

```

Indexing in torch

1. if x is of shape (N, D) , how priority of indexing works: dim N first, then dim D .
2. Example: $x[0]$ will return the second row of the array x , or equivalent to $x[0, :]$.

Code:

```

N = 3
T = 5

```

```
D = 10
x = torch.rand(N, T, D)
print(x[1].shape)
print(x[1, :, :].shape)
print((x[1] == x[1, :, :]).all())
```

Output:

```
torch.Size([5, 10])
torch.Size([5, 10])
tensor(True)
```

Helper functions for images

```
def preprocess(img, size=512):
    transform = T.Compose([
        T.Resize(size),
        T.ToTensor(),
        T.Normalize(mean=SQUEEZENET_MEAN.tolist(),
                    std=SQUEEZENET_STD.tolist()),
        T.Lambda(lambda x: x[None]),
    ])
    return transform(img)

def deprocess(img):
    transform = T.Compose([
        T.Lambda(lambda x: x[0]),
        T.Normalize(mean=[0, 0, 0], std=[1.0 / s for s in SQUEEZENET_STD.tolist()]),
        T.Normalize(mean=[-m for m in SQUEEZENET_MEAN.tolist()], std=[1, 1, 1]),
        T.Lambda(rescale),
        T.ToPILImage(),
    ])
    return transform(img)

def rescale(x):
    low, high = x.min(), x.max()
    x_rescaled = (x - low) / (high - low)
    return x_rescaled

def rel_error(x, y):
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))

def features_from_img(imgpath, imgsize):
    img = preprocess(PIL.Image.open(imgpath), size=imgsize)
    img_var = img.type(dtype)
    return extract_features(img_var, cnn), img_var
```

Extracting features

1. Using `cnn_model._modules.values()`

```
# We provide this helper code which takes an image, a model (cnn), and returns a list of
# feature maps, one per layer.
def extract_features(x, cnn):
    """
    Use the CNN to extract features from the input image x.

    Inputs:
    - x: A PyTorch Variable of shape (N, C, H, W) holding a minibatch of images that
      will be fed to the CNN.
    - cnn: A PyTorch model that we will use to extract features.

    Returns:
    - features: A list of feature for the input images x extracted using the cnn model.
      features[i] is a PyTorch Variable of shape (N, C_i, H_i, W_i); recall that features
      from different layers of the network may have different numbers of channels (C_i) and
      spatial dimensions (H_i, W_i).
    """
    features = []
    prev_feat = x
    for i, module in enumerate(cnn._modules.values()):
        next_feat = module(prev_feat)
        features.append(next_feat)
        prev_feat = next_feat
    return features

# -*- coding: utf-8 -*-
"""
Created on Tue Mar 31 15:08:13 2026

@author: reina
"""
import torch
import torchvision
from pprint import pprint

dtype = torch.float32
```

```

model = torchvision.models.resnet50()
cnn = torchvision.models.squeezenet1_1(pretrained=True)
features = cnn.features # Layers in cnn
print(features.type(dtype))

feats = []
N = 1; C=3; H=224; W=224 # properties of an image
prev_feat = torch.rand(N, C, H, W)
for i, module in enumerate(cnn._modules.values()):
    next_feat = module(prev_feat)
    feats.append(next_feat)
    prev_feat = next_feat
print(len(features))
print(feats[0].shape)

print(list(enumerate(cnn.parameters())) # weights

l = [(n, v.shape) for n, v in list(cnn.named_parameters())]
pprint(l, indent=2)

```

2. Note:

1. Layers of a net (inherited from `nn.Module`) are stored in `Module._modules`, which is initialized in `__construct`.
2. `self._modules` is updated in `__setattr__`. `__setattr__(obj, name, value)` is called when `obj.name = value` is executed.
3. If one runs a forward pass of a net inherited from `nn.Module`, the `nn.Module.__call__` will be called, in which `self.forward` is called. However, one has overrode the `forward` when implementing the net.

Pytorch Loss-Input Confusion

1. `torch.nn.functional.binary_cross_entropy` takes logistic sigmoid `nn.Sigmoid()` values as inputs.
2. `torch.nn.functional.binary_cross_entropy_with_logits` takes logits as inputs.
3. `torch.nn.functional.cross_entropy` takes logits as inputs (performs `log_softmax` internally, no `Softmax` needed just `nn.Linear` as final layer, `Softmax` is built-in into the loss function!)
4. `torch.nn.functional.nll_loss` is like `cross_entropy` but takes log-probabilities (log-softmax) `nn.Softmax(dim=1)` or `nn.LogSoftmax(dim=1)` values as inputs.

Tensor shapes

From Simple BEV repo: We maintain consistent axis ordering across all tensors. In general, the ordering is N, T, C, Z, Y, X, where

- N: batch
- T: Sequence (for temporal or multiview data)
- C: channels, or often also written as D
- Z: depth
- Y: height
- X: width

The ordering stands even if a tensor is missing some dims, for example, plain images are B, C, Y, X (as is the pytorch standard).

Axis directions

- Z: forward
- Y: down
- X: right

This means the top-left of an image is "0.0", and coordinates increase as you travel right and down. `z` increases forward because it's the depth axis.

<code>nn.Sequential</code>	<code>nn.ModuleList</code>

<code>model.named_children()</code>	<code>model.named_modules()</code>

detach()	requires_grad(False)
detaches a tensor from the computation graph, it creates a new tensor that shares the same data. However the new tensor is no longer being tracked by autograd.	temporarily disables gradient tracking by setting requires_grad flag to false.

pytorch-book

1. Chapter 2

```
1. import torch
   device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

   net.to(device)
   images = images.to(device)
   labels = labels.to(device)
   output = net(images)
   loss = criterion(output, labels)
```

Pytorch

Tensor

- From an interface perspective, operations on Tensors can be divided into two categories:
 - torch.function, such as torch.save, etc.
 - tensor.function, such as tensor.view, etc.
- For ease of use, most operations on Tensors support both types of interfaces simultaneously, and this book does not make a specific distinction, for example, torch.sum(a, b) and a.sum(b) are functionally equivalent.
- From a storage perspective, operations on Tensors can be divided into two categories:
 - A function that does not modify its own data, such as a.add(b) will return a new Tensor as the result of the addition.
 - A function that modifies its own data, such as a.add_(b) will still store the result if the addition in a, but a will be modified.
- Functions whose names end with an underscore are inplace functions, meaning they modify the caller's own data. This distinction is important in practical applications.

Creating Tensors

- There are many ways to create tensors in Pytorch, as shown in Table 3-1.

Function	Functionality
Tensor(*sizes)	Basic constructor
tensor(data,)	Constructor similar to np.array()
ones(*sizes)	Tensor with all ones
zeros(*sizes)	Tensor with all zeros
eye(*sizes)	The diagonal is 1, and the rest are 0
arange(s, e, step)	From s to e, the step size is step
linspace(s, e, steps)	From s to e, divide evenly into steps
rand/randn(*sizes)	uniform/standard distribution
normal(means, std)/uniform(from, to)	normal distribution/ uniform distribution
randperm(m)	random arrangement
tensor.new_*/torch.*_like	create a Tensor of the same size, filled with * type (e.g. tensor_new_ones, is filled with all ones), and with the same torch.dtype and torch.device

- All these creation methods allow you to specify the data type (dtype) and the storage device (CPU/GPU) during creation.
- The tensor function can be used to create a new Tensor. It can accept a list and create a new tensor based on the data in the list. It can also create a tensor based on the data in the list. It can also create a tensor based on a specified data shape and can pass in other tensors. Several examples will be given below.

4. It's important to note that when `t.Tensor(*sizes)` creates a Tensor, the system doesn't immediately allocate space. It only calculates whether there's enough remaining memory and allocates space immediately after tensor creation. In practice we recommend using `torch.tensor` instead of `torch.Tensor`. Let's compare the differences between the two.
5. `torch.Tensor` is a python class, and by default it is `torch.FloatTensor()`. Running `torch.Tensor([2, 3])` will directly call the `__init__()` constructor of the `Tensor` class, generating a single-precision floating-point Tensor(`FloatTensor`).
6. `torch.tensor` is a python function with the prototype `torch.tensor(data, dtype=None, device=None, requires_grad=False)`. `data` supports types such as list, tuple, array, and scalar. `torch.tensor()` directly copies data from `data` and generates the corresponding Tensor based on the original data type.
7. Since `torch.tensor()` can generate a corresponding Tensor based on the data type, we recommend using the `torch.tensor()` method to create a new Tensor in practical applications.

Types of Tensor

8. Tensor types can be further divided into device type and data type (`dtype`). Device type is divided into CUDA and CPU types, and numeric types include bool, float, and int.
9. Tensor data types are shown in Table 3-2, and each data type has CPU and GPU versions. Readers can obtain the device type of a tensor using `tensor.device` and the data type using `tensor.dtype`.
10. The default data type of a Tensor is `FloatTensor`. Readers can modify the default Tensor type using `torch.set_default_tensor_type` (if the default type is a GPPU Tensor, then all operations will be performed on the GPU) or `torch.set_default_dtype` (only accepts float input types).
11. Understanding the type of a Tensor is helpful for analyzing memory usage. For example, a `FloatTensor` of shape (1000, 1000, 1000) has $1000 \times 1000 \times 1000 = 10^9$ elements, each occupy $32\text{bit} \div 8 = 4\text{byte}$ of memory/GPU memory. `HalfTensor` is specifically designed for GPU versions. With the same number of elements, `HalfTensor` uses only half the GPU memory of a `FloatTensor`.
12. Therefore, using `HalfTensor` can greatly alleviate the problem of insufficient GPU memory. It should be noted that `HalfTensor` has limited numerical values and precision, which may lead to data overflow issues.
13. Tensor data type

Data Type	dtype	CPU tensor	GPU tensor
32-bit floating-point	<code>torch.float32</code> or <code>torch.float</code>	<code>torch.FloatTensor</code>	<code>torch.cuda.FloatTensor</code>
64-bit floating-point	<code>torch.float64</code> or <code>torch.double</code>	<code>torch.DoubleTensor</code>	<code>torch.cuda.DoubleTensor</code>
16-bit half-precision floating-point	<code>torch.float16</code> or <code>torch.half</code>	<code>torch.HalfTensor</code>	<code>torch.cuda.HalfTensor</code>
8-bit unsigned integer	<code>torch.uint8</code>	<code>torch.ByteTensor</code>	<code>torch.cuda.ByteTensor</code>
8-bit signed integer	<code>torch.int8</code>	<code>torch.CharTensor</code>	<code>torch.cuda.CharTensor</code>
16-bit signed integer	<code>torch.int16</code> or <code>torch.short</code>	<code>torch.ShortTensor</code>	<code>torch.cuda.ShortTensor</code>
32-bit signed integer	<code>torch.int32</code> or <code>torch.int</code>	<code>torch.IntTensor</code>	<code>torch.cuda.IntTensor</code>
64-bit signed integer	<code>torch.int64</code> or <code>torch.long</code>	<code>torch.LongTensor</code>	<code>torch.cuda.LongTensor</code>
Boolean type	<code>torch.bool</code>	<code>torch.BoolTensor</code>	<code>torch.cuda.BoolTensor</code>

7. Common methods for converting between different types of tensors are as follows:

1. The most common approach is `tensor.type(new_type)`, but there are also shortcut methods such as `tensor.float()`, `torch.long()`, and `tensor.half()`.
2. Conversion between CPU tensors and GPU tensors is achieved using `tensor.cuda()` and `tensor.cpu()`, and `tensor.to(device)` can also be used.
3. Creating tensors of the same type: `torch.*_like` and `torch.new_*`. These two methods are suitable for writing device compatible code:
 1. `torch.*_like(tensorA)` generates a new tensor with the same attributes as `tensorA` (such as type, shape and CPU/GPU)
 2. `tensor.new_*(new_shape)` creates a new tensor with a different shape but the same attributes.

References:

1. Pytorch book
2. Modern Computer Vision with PyTorch: Explore deep learning concepts and implement over 50 real-world image applications, Kishore Ayyadevara.

3. [How pytorch's nn.Module register submodule, stackoverflow](#)